



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS
Facultad de Ingeniería

Manual Técnico del Portal del Laboratorio de Archivística y Humanidades Digitales

Manual técnico

AUTORES:

Equipo del Laboratorio de Archivística
Universidad Distrital Francisco José de
Caldas

Índice general

1	Introducción y alcance	3	
1.1	Público objetivo	3	
1.2	Convenciones	4	
2	Visión general de la arquitectura	5	
2.1	Contenedores del despliegue	6	
2.2	Red y comunicación entre contenedores	6	
3	Stack tecnológico	7	
3.1	Frontend (Astro)	7	
3.1.1	Ejecución local del frontend	7	
3.2	Contenedor del frontend	8	
3.3	Stack OrfeoNG	8	
3.4	Stack DSpace 10	9	
3.5	Apache Tika	9	
4	Estructura del repositorio	10	
4.1	Páginas del portal	10	
5	Despliegue local	11	
5.1	Requisitos	11	
5.2	Variables de entorno	11	
5.3	Levantar el portal completo	12	
5.4	Verificación del despliegue	12	
6	Configuración del proxy nginx	14	
6.1	Recargar la configuración del proxy	14	
7	Gestión de contenido	16	
7.1	Noticias	16	
7.1.1	Agregar una noticia	17	
7.1.2	Script de actualización	18	
7.1.3	Orden de las noticias	18	
7.2	Equipos	18	
7.2.1	Agregar o modificar un equipo	19	
7.3	Salas	20	
7.4	Software	20	
7.4.1	Catálogo de software (/software)	21	
7.4.2	Listado de software de la portada	21	
7.5	Investigación	22	
7.6	Servicios de la portada	22	
7.7	Flujo de publicación de cambios	22	
8	Mantenimiento y operación	23	
8.1	Levantar el frontend	23	

8.2	Gestor de los tres stacks	23
8.3	Registros (logs)	23
8.4	Respaldos	24
8.4.1	Ejemplo de respaldo de un volumen	24
8.4.2	Actualización de imágenes base	25
8.5	Actualizar el código del repositorio	25
8.6	Monitoreo básico	26
9	Resolución de problemas	28
10	Seguridad y buenas prácticas	29
10.1	Buenas prácticas de gestión de contenido	29

1 Introducción y alcance

El presente manual técnico documenta la arquitectura, el despliegue, la gestión de contenido, el mantenimiento y la resolución de problemas del **portal web del Laboratorio de Archivística y Humanidades Digitales** de la Universidad Distrital Francisco José de Caldas.

El alcance de este documento se limita a los componentes que se encuentran versionados en el repositorio **Lab-Archivística-Frontend**:

- Frontend desarrollado con **Astro** (sitio estático generado).
- Proxy **nginx** global que publica el portal en el puerto 80.
- Stack **OrfeoNG** (gestión documental).
- Stack **DSpace 10** (repositorio digital).
- Servicio **Apache Tika** (extracción de metadatos).
- Scripts de despliegue, operación y actualización de contenido.
- Manuales de usuario y técnico (directorio **docs/**).

Las aplicaciones **Archivematica**, **AtoM** y **ExifTool** se despliegan fuera de este repositorio (en el servidor host) y se integran mediante el proxy. Este manual las menciona a nivel de integración, pero no documenta su instalación interna.

1.1 Público objetivo

Este manual está dirigido a:

- **Administradores del portal**: responsables de desplegar, actualizar y mantener el sitio.

- **Gestores de contenido:** personas encargadas de publicar noticias y mantener los catálogos de equipos, salas y software.
- **Desarrolladores:** quienes modifican el código del frontend.

1.2 Convenciones

- Los comandos de terminal aparecen en cajas de **Terminal**.
- Los fragmentos de código o configuración aparecen en cajas de **Código**.
- Las advertencias y recomendaciones aparecen en cajas resaltadas.
- **IMPORTANTE:** La ejecución de un script de bash en el manual se indica bash nombre_comando, la palabra bash hace referencia a: ./nombre_comando , tomar siempre esto en cuenta a la hora de ejecutar un script.

2 Visión general de la arquitectura

El portal sigue un modelo de **microservicios por contenedores** orquestado con Docker Compose. Un único proxy **nginx** (**lab-proxy**) actúa como puerta de entrada pública y distribuye el tráfico hacia cada aplicación según la ruta solicitada.

Diagrama de arquitectura

```
Usuario ->nginx (lab-proxy, puerto 80)

/ ->astro-frontend (4321)

/orfeo/... ->orfeo-nginx (80)

/dspace/ ->dspace-frontend (4000)

/dspace/server/ ->dspace-api (8080)

/tika/api/ ->apache-tika (9998)

/atom/ ->host.docker.internal:63001 (AtoM)

/archivematica/ ->host.docker.internal:63002

/exiftool/ ->host.docker.internal:63003
```

2.1 Contenedores del despliegue

Contenedor	Imagen / Base	Puerto expuesto
lab-proxy	nginx:1.25-alpine	80
astro-frontend	node:22-alpine + serve	127.0.0.1:4321
orfeo-mysql	mysql:5.7	127.0.0.1:3307
orfeo-backend	build local	interno
orfeo-frontend	build local	interno
orfeo-sockets	build local	interno
orfeo-nginx	nginx:1.25-alpine	127.0.0.1:64000
dspace-db	dspace-postgres-loadsql:10	interno
dspace-solr	dspace-solr:10	interno
dspace-api	dspace:dspace-10_x	127.0.0.1:8080
dspace-frontend	dspace-angular:10	127.0.0.1:4000
apache-tika	apache/tika:latest-full	127.0.0.1:9998

Nota: los puertos 3307, 4321, 64000, 8080, 4000 y 9998 están vinculados únicamente a 127.0.0.1; el único puerto expuesto al exterior es el 80 del proxy. Las aplicaciones Orfeo, DSpace y Tika se publican vía lab-proxy.

2.2 Red y comunicación entre contenedores

Todos los contenedores del despliegue pertenecen a la red Docker `lab-network` y se comunican entre sí por su nombre de servicio. Las aplicaciones externas (AtoM, Archivematica, ExifTool) se alcanzan a través de `host.docker.internal`, que resuelve la IP del host.

- `astro-frontend` sirve el sitio estático generado por Astro (carpeta `dist`) usando `serve` en el puerto 4321.
- `lab-proxy` recibe el tráfico del puerto 80 y lo reenvía según la tabla de rutas del capítulo 6.
- `orfeo-nginx` sirve como proxy interno del stack OrfeoNG (backend PHP + frontend + sockets).
- `dspace-api` ejecuta la migración de base de datos al arrancar y expone la API REST de DSpace en el puerto 8080.
- `dspace-frontend` sirve la interfaz Angular 19 de DSpace en el puerto 4000.

3 Stack tecnológico

3.1 Frontend (Astro)

El portal es una aplicación **Astro** con componentes **React**:

- Astro 7.1.6.
- React 19.
- Tailwind CSS v4 (paleta personalizada: **navy** y **accent**).
- TypeScript en modo estricto.
- `@tailwindcss/typography` para el contenido de las noticias.
- **Content collections** con el cargador `glob` para leer las noticias desde `public/news/`.
- **Astro transitions** (`ClientRouter`) para las transiciones entre páginas.

3.1.1 Ejecución local del frontend

Instalación de dependencias:

```
Terminal:  
npm install
```

En modo desarrollo (servidor con recarga en caliente):

```
Terminal:  
npm run dev
```


Para generar el sitio estático y previsualizarlo:

Terminal:

```
npm run build  
npm run preview
```

El sitio generado queda en `dist/` y es lo que sirve el contenedor `astro-frontend`.

3.2 Contenedor del frontend

El Dockerfile usa `node:22-alpine` para construir el sitio y sirve la carpeta `dist` con `serve` en el puerto 4321:

Etapas del Dockerfile

```
FROM node:22-alpine AS build  
  
# ... npm ci + npm run build ...  
  
FROM node:22-alpine  
  
# ... npm install -g serve ...  
  
CMD ["serve", "dist", "1", "4321"]
```

3.3 Stack OrfeoNG

El stack OrfeoNG está compuesto por:

- `orfeo-mysql`: base de datos MySQL 5.7 (volumen `orfeo_mysql_data`).
- `orfeo-backend`: API PHP del sistema.
- `orfeo-frontend`: interfaz de usuario.
- `orfeo-sockets`: servidor de WebSockets para notificaciones.
- `orfeo-nginx`: proxy interno del stack.

Los repositorios de OrfeoNG se clonan en tiempo de construcción desde GitHub (variables `BACKEND_REPO`, `FRONTEND_REPO`, `SOCKETS_REPO`).

3.4 Stack DSpace 10

- `dspace-db`: PostgreSQL con carga inicial (SQL de DSpace 10).
- `dspace-solr`: índices Solr (búsqueda y descubrimiento).
- `dspace-api`: API REST de DSpace (Java, puerto 8080).
- `dspace-frontend`: interfaz Angular 19 (puerto 4000).

Los datos persistentes viven en los volúmenes `dspace_pgdata`, `dspace_solr_data` y `dspace_assetstore`.

3.5 Apache Tika

El contenedor `apache-tika` usa la imagen `apache/tika:latest-full` y expone la API en el puerto 9998 (solo 127.0.0.1). La interfaz web estática se sirve desde `tika-ui/` a través del proxy.

4 Estructura del repositorio

Ruta	Contenido
src/pages/	Páginas Astro (Inicio, Equipos, Software, Salas, Noticias, Investigación)
src/components/	Componentes (Hero, Footer, SoftwareEquipment, etc.)
src/layouts/	Layout principal (encabezado, pie y metadatos)
src/data/equipos.ts	Datos centralizados de equipos
src/content/	Script actualizar_pagina-noticias.sh
src/content.config.ts	Esquema de las content collections
public/news/	Noticias en Markdown (content collection)
public/images/news/	Imágenes de noticias
public/	Archivos estáticos (logos, favicon, robots.txt, sitemap)
src/styles/global.css	Estilos globales y paleta de colores
nginx/default.conf	Configuración del proxy
docker-compose.yml	Orquestación de todos los servicios
orfeo/	Código y configuración de OrfeoNG
dspace/	Configuración de DSpace (local.cfg, solr-init.sh)
tika-ui/	Interfaz estática de Tika
Levantar_Pagina.sh	Rebuild del frontend
gestor-app-lab.sh	Levantar/derribar los 3 stacks
docs/	Manuales (usuario y técnico) en LaTeX

4.1 Páginas del portal

Ruta	Contenido
/	Página de inicio (carrusel, servicios, noticias, software, proceso, contacto)
/equipos	Catálogo de equipos con filtros
/equipos/<id>	Ficha de detalle de un equipo
/software	Catálogo de software con búsqueda y filtros
/servicios	Vista general de servicios
/servicios/salas	Listado de salas
/servicios/salas/<id>	Ficha de detalle de una sala
/noticias	Listado de noticias
/noticias/<slug>	Página de detalle de una noticia
/investigacion	Proyectos y líneas de investigación
/404	Página de error 404

5 Despliegue local

5.1 Requisitos

- Docker Engine 24+ y Docker Compose v2.
- Node.js 22 (solo para desarrollo del frontend).
- Git.
- Conexión a internet para la descarga de imágenes y repositorios de Orfeo (que se clonan en tiempo de construcción).

5.2 Variables de entorno

La configuración sensible se define mediante variables de entorno (archivo `.env` en la raíz). Las principales son:

- `MYSQL_ROOT_PASSWORD`, `DB_NAME`, `DB_USER`, `DB_PASS`: credenciales de MySQL de Orfeo.
- `AES_KEY`: clave de cifrado compartida por Orfeo.
- `SERVER_URL`: URL pública de Orfeo (por defecto `http://localhost:64000`).
- `PORT`: puerto host de `orfeo-nginx` (por defecto 64000).
- `BACKEND_REPO`, `FRONTEND_REPO`, `SOCKETS_REPO`: repositorios GitHub de OrfeoNG.

Importante: el archivo `.env` NO debe versionarse en el repositorio público. Copie `.env.example` (si existe) y ajuste los valores.

Otras variables de entorno, importantes:

- `dspace.server.url`: Locación del servicio de servidor de Dspace.
- `dspace.ui.url`: Locación del servicio de frontend de Dspace.
- `cors.allowed.origins` / `rest.cors.allowed-origins`: Ubicaciones que permite dspace para ser accedido desde ellas, necesitan cambiar junto con el certificado del sitio si se requiere.

5.3 Levantar el portal completo

Terminal:

```
docker compose up -d
```

Para construir las imágenes (primera vez o tras cambios en Dockerfile):

Terminal:

```
docker compose build
```

Para detener todos los servicios:

Terminal:

```
docker compose down
```

Para ver el estado de los contenedores:

Terminal:

```
docker compose ps
```

5.4 Verificación del despliegue

1. Espere a que todos los contenedores estén en estado `running/healthy`:

Terminal:

```
docker compose ps
```

2. Verifique el portal principal:

Terminal:

```
curl -I http://localhost
```

3. Verifique las rutas de Orfeo, DSpace y Tika:

Terminal:

```
curl -I http://localhost/orfeo/  
curl -I http://localhost/dspace/  
curl -I http://localhost/tika/
```

4. Revise los logs del proxy si alguna ruta falla:

Terminal:

```
docker compose logs -f lab-proxy
```

6 Configuración del proxy nginx

El archivo `nginx/default.conf` define las rutas del portal. A continuación se resume la distribución del tráfico:

Ruta	Destino	Observación
/	astro-frontend:4321	Portal principal
/orfeo/	orfeo-nginx:80	UI de Orfeo
/orfeo-ng/	orfeo-nginx:80	UI NG de Orfeo
/dspace/	dspace-frontend:4000	UI Angular de DSpace
/dspace/server/	dspace:8080	API REST de DSpace
/server/	dspace:8080	Compatibilidad con instalaciones previas
/ng_backend/	orfeo-nginx:80	API de Orfeo
/socket/, /socket.io/	orfeo-nginx:80	WebSockets de Orfeo
/tika/	tika-ui (estática)	Interfaz de Tika
/tika/api/	apache-tika:9998	API de Tika
/atom/	host.docker.internal:63001	AtoM (externo)
/archivematica/	host.docker.internal:63002	Archivematica (externo)
/exiftool/	host.docker.internal:63003	ExifTool (externo)

Importante: los bloques `/atom/`, `/archivematica/` y `/exiftool/` dependen de `host.docker.internal`, lo que exige que las aplicaciones correspondientes estén corriendo en el servidor host. Si alguna no está disponible, se generarán errores 502 en esas rutas.

Para las rutas de Orfeo se usa un **resolver** de Docker (127.0.0.11) con nombres de upstream variables; esto permite recargar la configuración sin reiniciar el proxy cuando cambian las IPs de los contenedores.

6.1 Recargar la configuración del proxy

Después de modificar `nginx/default.conf`, se recarga la configuración sin detener el contenedor:

Terminal:

```
docker compose exec lab-proxy nginx -s reload
```

Para validar la sintaxis de la configuración antes de recargar:

Terminal:

```
docker compose exec lab-proxy nginx -t
```

Recomendación: pruebe siempre con `nginx -t` antes de recargar para evitar dejar el portal caído por un error de sintaxis.

7 Gestión de contenido

La página se alimenta de distintas fuentes dentro del repositorio. Este capítulo explica cómo agregar o modificar cada tipo de contenido: noticias, equipos, salas, software e investigación. Salvo el script de noticias, los cambios se aplican editando el código fuente y reconstruyendo el frontend con `Levantar_Pagina.sh` (Capítulo 8) o, en desarrollo, con `npm run build`.

7.1 Noticias

Las noticias se escriben como archivos **Markdown** con portada (**frontmatter**) y se almacenan en `public/news/`. El esquema se define en `src/content.config.ts`:

`content.config.ts`

```
import { defineCollection, glob } from 'astro:content';

const news = defineCollection({

  loader: glob({

    pattern: '**/[^_]*.md',

    base: './public/news',

  }),

  schema: z.object({

    title: z.string(),

    date: z.coerce.date(),
```

```
excerpt: z.string(),

author: z.string().default('GLUD'),

image: z.object({ src: z.string(), alt: z.string() }).optional(),

}),

});

export const collections = { news };
```

7.1.1 Agregar una noticia

1. Crear un archivo AAAA-MM-DD-slug.md en public/news/. El nombre del archivo define la fecha de publicación y la URL de la noticia (/noticias/AAAA-MM-DD-slug/).
2. Definir el frontmatter: title, date, excerpt, author y, opcionalmente, image.
3. Escribir el cuerpo en Markdown (se renderiza con @tailwindcss/typography).
4. Ejecutar el script de actualización para regenerar la página.

Ejemplo real de una noticia publicada:

```
public/news/2026-07-30-dspace10-disponible.md

--

title: "Nuevo DSpace 10 disponible y funcional"

date: 2026-07-30

excerpt: .E1 repositorio digital del laboratorio ha sido actualizado a
DSpace 10."

author: "Nicolas Acevedo"

image: /images/news/dspace.png"

--
```

Las imágenes se ubican en public/images/news/ y se referencian con la ruta absoluta /images/news/<archivo>.

Nota: la imagen es opcional. Si se omite, la tarjeta de la noticia no muestra imagen. Se recomienda PNG o SVG con fondo transparente para una mejor visualización.

7.1.2 Script de actualización

El script `src/content/actualizar_pagina-noticias.sh` reconstruye la imagen `astro-frontend` y recrea el contenedor (sin reiniciar el resto de servicios):

Terminal:

```
bash src/content/actualizar_pagina-noticias.sh
```

Con reconstrucción limpia (sin caché):

Terminal:

```
bash src/content/actualizar_pagina-noticias.sh -no-cache
```

7.1.3 Orden de las noticias

Las noticias se ordenan de forma descendente por su campo `date`. La página de inicio muestra las 4 más recientes y el carrusel las 3 más recientes. El listado completo (`/noticias`) las muestra todas.

7.2 Equipos

Los equipos se definen de forma centralizada en `src/data/equipos.ts`. Este archivo alimenta la sección de la portada, el listado en `/equipos` y la ficha de detalle en `/equipos/<id>`:

```
src/data/equipos.ts
```

```
export interface Equipo {  
  
  id: number;
```

```
name: string;

description: string;

available: boolean;

category: string;

}

export const equipos: Equipo[] = [

{

id: 1,

name: 'Computadores',

description: 'Equipo de cómputo de alta gama para procesamiento de datos y digitalización documental.',

available: true,

category: 'Cómputo',

},

];
```

7.2.1 Agregar o modificar un equipo

1. Abra `src/data/equipos.ts`.
2. Para agregar: añada un nuevo objeto al arreglo con un `id` consecutivo (los `ids` no deben repetirse, ya que definen la URL `/equipos/<id>`).
3. Para modificar: edite los campos del objeto correspondiente.
4. Para cambiar la disponibilidad: ajuste `available` a `true` o `false`.
5. Reconstruya el frontend para publicar los cambios.

Nota: a diferencia de las noticias, los cambios en equipos, salas y software requieren reconstruir el frontend para verse publicados.

7.3 Salas

Las salas están definidas en el arreglo `salas` de `src/pages/servicios/salas/[id].astro`:

```
src/pages/servicios/salas/[id].astro
```

```
const salas = [  
  
  { id: 1, name: 'Sala de Digitalización', capacity: 6, available: true,  
    description: '...' },  
  
  { id: 2, name: 'Sala de Trabajo Colaborativo', capacity: 12, available:  
    true, description: '...' },  
  
  { id: 3, name: 'Cubículo de Estudio', capacity: 2, available: false,  
    description: '...' },  
  
];
```

Cada sala define `id`, `name`, `capacity`, `available` y `description`. Al agregar o eliminar salas debe actualizarse también `getStaticPaths()` con los `id` disponibles:

```
getStaticPaths
```

```
export async function getStaticPaths() {  
  
  return [  
  
    { params: { id: '1' } },  
  
    { params: { id: '2' } },  
  
    { params: { id: '3' } },  
  
  ];  
  
}
```

7.4 Software

7.4.1 Catálogo de software (/software)

El catálogo completo se define en el arreglo `softwares` de `src/pages/software/index.astro`. Cada entrada define:

- `id`: identificador numérico.
- `name`: nombre de la herramienta.
- `version`: versión mostrada.
- `category`: categoría (Repositorio Digital, Preservación Digital, Descripción Archivística, Gestión Documental, Extracción de Metadatos, Metadatos).
- `licenseType`: tipo de licencia.
- `description`: descripción breve.
- `url`: URL de acceso (opcional; si se omite, la tarjeta no muestra botón Acceder).

7.4.2 Listado de software de la portada

El listado de la portada se define en `src/components/SoftwareEquipment.astro` en el arreglo `softwareList`. Cada entrada define `name`, `href` y `logo` (un SVG embebido):

```
src/components/SoftwareEquipment.astro
```

```
const softwareList = [  
  
  { name: 'DSpace', href: '/dspace/', logo: '<svg ...>' },  
  
  { name: 'Archivematica', href:  
    'http://labhumanidades.udistrital.edu.co:62080', logo: '<svg ...>' },  
  
];
```

El `href` puede ser una ruta interna servida por el proxy nginx (Capítulo 6) o una URL externa. El `logo` es un SVG embebido que se renderiza con `set:html`.

7.5 Investigación

Los proyectos de investigación se definen en el arreglo embebido de `src/pages/investigacion/index.as`. Cada tarjeta define:

- **title:** título del proyecto.
- **desc:** descripción breve.
- **tag:** etiqueta de estado (En curso, Investigación, Publicado).
- **color:** color de la etiqueta (green, blue, accent).

Para agregar un proyecto, añada un objeto al arreglo y reconstruya el frontend.

7.6 Servicios de la portada

Las tarjetas de servicios de la portada se definen en `src/components/Services.astro` (Software especializado, Laboratorios Virtuales, Préstamo de equipos, Préstamos de sala). Cada tarjeta define `title`, `description`, `icon` (SVG) y `href`.

7.7 Flujo de publicación de cambios

1. Edite el archivo correspondiente (Markdown de noticia, `equipos.ts`, componente de salas/software/investigación).
2. Pruebe en desarrollo:

Terminal:

```
npm run dev
```

3. Genere el sitio:

Terminal:

```
npm run build
```

4. Verifique en `dist/` que las páginas nuevas se generaron.
5. En el servidor, actualice el repositorio con `git pull` y reconstruya el frontend.

8 Mantenimiento y operación

8.1 Levantar el frontend

El script `Levantar_Pagina.sh` reconstruye únicamente el contenedor `astro-frontend` tras cambios en el frontend:

Terminal:

```
bash Levantar_Pagina.sh
```

Para forzar una construcción limpia:

Terminal:

```
bash Levantar_Pagina.sh -no-cache
```

8.2 Gestor de los tres stacks

El script `gestor-app-lab.sh` opera sobre los tres directorios del servidor (`Lab-Archivistica-Frontend`, `archivematica/hack` y `atom`):

Terminal:

```
bash gestor-app-lab.sh up  
bash gestor-app-lab.sh down
```

8.3 Registros (logs)

Registros por contenedor

```
docker compose logs -f astro-frontend
```

```
docker compose logs -f orfeo-backend
```

```
docker compose logs -f dspace
```

```
docker compose logs -f apache-tika
```

```
docker compose logs -f lab-proxy
```

Para ver los últimos N registros sin seguir:

Terminal:

```
docker compose logs -tail 100 astro-frontend
```

8.4 Respaldos

Los datos persistentes viven en volúmenes Docker:

- orfeo_mysql_data: base de datos de Orfeo.
- dspace_pgdata: base de datos de DSpace.
- dspace_solr_data: índices de Solr.
- dspace_assetstore: objetos del repositorio.

Recomendación: realice respaldos periódicos de estos volúmenes (p. ej. con `docker run -rm -v <volumen>:/data -v $(pwd):/backup busybox cp -r /data /backup`) y guárdelos fuera del servidor.

8.4.1 Ejemplo de respaldo de un volumen

Respaldo del volumen de Orfeo

```
docker run -rm -v orfeo_mysql_data:/data  
  
-v /ruta/respaldo:/backup busybox  
  
cp -r /data /backup/orfeo-mysql
```

8.4.2 Actualización de imágenes base

Para actualizar las imágenes base (nginx, node, DSpace, Tika):

1. Haga un respaldo de los volúmenes de datos.
2. Actualice las versiones en `docker-compose.yml`.
3. Reconstruya:

Terminal:

```
docker compose build
```

4. Reinicie:

Terminal:

```
docker compose up -d
```

5. Verifique el estado y los datos después del reinicio.

8.5 Actualizar el código del repositorio

1. En el servidor, dentro de Lab-Archivistica-Frontend:

Terminal:

```
git pull
```

2. Si hubo cambios en `public/news/`: ejecute

Terminal:

```
bash src/content/actualizar_pagina-noticias.sh
```

3. Si hubo cambios en el código del frontend (componentes, datos, estilos):

Terminal:

```
bash Levantar_Pagina.sh
```

4. Si hubo cambios en `docker-compose.yml` o en el `Dockerfile`:

Terminal:

```
docker compose up -d -build
```

5. Si hubo cambios en `nginx/default.conf`: valide y recargue el proxy.

8.6 Monitoreo básico

- Estado de los contenedores:

Terminal:

```
docker compose ps
```

- Uso de recursos:

Terminal:

```
docker stats
```

- Espacio en disco:

Terminal:

```
df -h
```

- Verificación de respuesta del portal:

Terminal:

```
curl -I http://localhost
```

9 Resolución de problemas

- **Nginx no me redirige a donde necesito.** Detenga docker, y cambie el mapeo en `docker-compose.yml`, por la dirección necesitada incluída las variables necesarias.
- **El puerto 80 está ocupado.** Detenga el servicio que lo usa o cambie el mapeo en `docker-compose.yml`.
- **Error 502 en /atom/, /archivematica/ o /exiftool/.** Verifique que la aplicación externa esté corriendo en el host y que `host.docker.internal` resuelva.
- **Orfeo no responde.** Confirme que `orfeo-mysql` esté sano (`docker compose ps`) y revise los logs del backend.
- **DSpace tarda en iniciar.** La API ejecuta una migración de base de datos al arrancar; espere a que el healthcheck del contenedor pase.
- **DSpace no carga al cambiar la dirección o certificado del sitio.** Revise dentro de la carpeta de dspace, el archivo `local.cfg` y cambie lo necesario, después vuelva a levantar el servicio. de datos al arrancar; espere a que el healthcheck del contenedor pase.
- **Noticias desactualizadas.** Ejecute `actualizar_pagina-noticias.sh` para reconstruir el frontend.
- **Los contenedores no resuelven entre sí.** Asegúrese de que todos pertenecen a la red `lab-network`.
- **El sitio muestra una versión antigua tras un cambio.** Reconstruya la imagen sin caché (`Levantar_Pagina.sh -no-cache`) y reinicie el contenedor.
- **Una imagen de noticia no aparece.** Verifique que el archivo exista en `public/images/news/` y que la ruta en el `frontmatter` comience con `/images/news/`.
- **La página de un equipo devuelve 404.** El `id` en `getStaticPaths()` (o en el arreglo `equipos`) no coincide con la URL solicitada; reconstruya el sitio.
- **El proxy recibe 502 en todas las rutas.** Verifique que `lab-proxy` y los servicios de destino estén en la misma red y en ejecución.

10

Seguridad y buenas prácticas

- No exponga los puertos internos (3307, 8080, 4000, 9998, 64000) a la red pública; mantenga el mapeo a 127.0.0.1.
- Proteja las credenciales en el archivo `.env`; no lo versiones en el repositorio público.
- Actualice periódicamente las imágenes base (`nginx:1.25-alpine`, `node:22-alpine`, DSpace, Tika) para corregir vulnerabilidades.
- Active HTTPS (certificado del servidor) para cifrar el tráfico del portal en producción.
- Haga respaldos automáticos de los volúmenes de datos y pruebe su restauración.
- Restrinja el acceso a los formularios de reserva y las cuentas de administrador de cada herramienta (DSpace, Orfeo, Archivematica, AtoM).
- Documente en este manual cualquier cambio de arquitectura relevante.

10.1 Buenas prácticas de gestión de contenido

- Verifique siempre la ortografía y el formato en `npm run dev` antes de publicar una noticia.
- Use nombres de archivo `AAAA-MM-DD-slug.md` coherentes para las noticias.
- Mantenga los `id` de equipos y salas estables (no reutilice `ids` eliminados en el corto plazo para evitar enlaces rotos).
- Comprima y optimice las imágenes de noticias (PNG/SVG recomendado).
- Después de cada publicación, verifique el sitio en producción.